University of North Georgia

Computer Science & Cyber Security

CSCI 3250 Computer Security

General policies: Work in teams of two students. Working as a solo cuts 10 points. Each student submits the same file.

File Encryption with Statistical Insights

Summary:

This Project involves developing a Python application that performs AES encryption and decryption on different type of files, while also extracting and reporting statistical information from both the original and encrypted data.

Students will implement secure file handling using cryptographic libraries (e.g., cryptography), validate encryption integrity, and analyze file characteristics using the mean and the standard deviation. The goal is to understand both how encryption protects data and how it transforms its structure, with attention to confidentiality and statistical analysis principles.

Learning Objectives and Outcomes:

By completing this project, students will be able to:

Technical Skills:

- Implement AES symmetric encryption using Python (the cryptography library)
- Apply secure key and IV generation practices
- Write Python code to perform file I/O, binary and hex manipulation.
- Generate and parse encrypted binary files
- Validate decrypted file content for integrity

Analytical Skills:

- Compute statistics in plaintext and ciphertext

- Calculate the mean and the standard deviation to evaluate similarities and differences in different type of files and encrypted files.
- Plot frequency histograms using matplotlib (optional 20 extra credit points)

Security Awareness:

- Understand the role of encryption in confidentiality
- Recognize how ciphertext differs statistically from plaintext

Expected Outputs:

- Print the first 32 bytes of each file and look for similarities and differences.
- Mean and standard deviation for Microsoft docx and xlsx files, pdf, text, png or JPED files.
- Mean and standard deviation for the same files encrypted.

Implementation.

- Create a directory called mydata that will contain 5 subdirectories that will hold at least 3 files from docx, xlsx, pdf, png/jped, txt. For example, the directory mydocx contains the files primer.docx, students.docx and contract.docx. The directory myxlsx will contain at least three spreadsheets and so on.
- Code in python a routine/method that will read each file from each directory and print the first 32 bytes with the file name:

```
import os as os
import binascii
import numpy as np

TO BE COMPLETED

for filetype in files:
  for filename in files[filetype]:
    with open(f'data/{filetype}/{filename}', 'rb') as f:
      chunk = f.read(32)
      print(f'{binascii.hexlify(chunk)} -- {filename}')
```

- Code in python a routine/method that will return the mean and standard deviation of the content of a file.:

```
def get_file_stats(filename):
  with open(filename, 'rb') as f:
    file_bytes = bytearray(f.read())
  data = np.frombuffer(file_bytes, np.uint8)
  return (np.mean(data), np.std(data))
```

- Code in python a routine/method that will print the mean and standard deviation of the content of each file. Use the method get_file_stats(filename).

- Code in python a routine/method that will encrypt a file and return the statistics of the file encrypted:

```
def encrypt_and_stats(filename):
  with open(filename, 'rb') as f:
    file_bytes = bytearray(f.read())
  key = os.urandom(32)
  iv = os.urandom(32)
  aes_cipher = Cipher(algorithms.AES(key), modes.GCM(iv))
  encryptor = aes_cipher.encryptor()
  ciphertext = encryptor.update(file_bytes) + encryptor.finalize()
  data = np.frombuffer(ciphertext, np.uint8)
  return(np.mean(data), np.std(data))
```

- Code in python a routine/method that will print the mean and standard deviation of the content of each file encrypted. Use the method encrypt_and_stats(filename).
- You must follow standard of commenting the python code, including the one given in this description. It is not allowed to input this code into an AI/internet driven software looking for help.
- Answer the following questions:
    - Any similarities in the first 32 bytes of data in each file. Explain
    - Any similarities in the statistics of the content of each file. Explain.

- Any similarities in the statistics of the content of each file encrypted. Explain.

What to submit:

1. A **pdf/Word file** that contains the following
   a. A cover sheet with
      i. Project number and name,
      ii. Course number, name, year, and semester,
      iii. Author's names and email addresses
      iv. Day and time
   b. A summary essay about the project in your own words, covering the following:
      i. The nature of the problem to be solved,
      ii. A description of how you solved this problem.
      iii. The issues and problems that you encountered and how did you solve those problems,
      iv. The concepts that you learned,
      v. Answers to the questions,
      vi. References as appropriate. If there are no references, then you indicate explicitly the use of no references.
      vii. Screenshots that show your findings.
2. The .py code as text files, i.e., .py.txt.

Grading Criteria:

Please read the Section What to Submit.

1. Submitting the project as a pdf/Word file:
   a. Cover sheet is 5 points.
   b. Summary essay 20 points including questions.
   c. Evidence of programs running – Screenshots, 30 points
   d. Each student must submit the same pdf file in D2L. If pdf files submitted are different, the first one in the list will be the one considered. If a team member does not submit the pdf file, 30 points will be cut to that student.
   e. Each student has a grade according to the fulfillment of responsibilities. If a student does not collaborate the student get

zero and his/her partner must complete the whole project.

2. Submitting of python code 30 points. No errors, no warnings. Each warning/error cut 10 points.
3. There are two slip days for submitting the project.
4. If more than 2 files are submitted will cut 10 points.
5. Presentation of this project in class. 20 points. Students need to be prepared for running the code in class and present results to classmates.

Note: for the presentation at the end of the semester, students must be aware that the code needs to be workable. i.e., the code must run.